

# AI 系统设计面试题总结

## 面试官真正想考什么

AI 系统设计题一般不会满足于“我用 LangChain 搭一个 RAG”。面试官更想看你是否具有生产级架构意识。

| 考察方向            | 面试官想确认什么             | 常见和分点            |
|-----------------|----------------------|------------------|
| 整体架构            | 你能否把 AI 应用拆成清晰分层     | 上来就讲框架，不讲链路和边界   |
| 模型网关            | 你是否知道模型调用需要统一治理      | 业务代码直接耦合供应商 API  |
| Prompt/Context  | 你是否知道提示词和上下文要版本化、可回放 | Prompt 写死在代码里    |
| RAG/Memory/Tool | 你是否能区分知识、记忆和真实业务动作   | 把所有上下文混在一起塞给模型   |
| 可观测与评测          | 你是否能证明系统质量变化         | 只靠人工试几条问题        |
| 安全合规            | 你是否知道模型不能绕过业务权限      | 只靠 Prompt 防越权和注入 |

系统设计题最怕空泛。好的回答要能沿着一次请求说清楚：用户请求进来后，经过哪些模块，每个模块解决什么问题，出了问题怎么定位，质量下降怎么回滚。

## 生产级 AI 应用架构

参考文章：《AI 应用系统设计：从 Prompt Demo 到生产级架构》

这一组题是 AI 系统的核心。你要能把 AI 应用拆成多个工程模块，而不是只说“前端发请求，后端调模型”。

建议掌握这些关键点：

- Prompt Demo 证明的是模型能回答，生产系统要证明的是系统能长期、稳定、可控地回答。
- 入口层负责鉴权、租户、限流、参数校验和请求分类。
- 编排层负责判断任务类型，是普通问答、RAG、Agent、多工具任务，还是异步批处理。
- Prompt/Context 层负责模板版本、变量校验、历史信息、检索证据、用户画像和工具说明。
- RAG 管共享知识，Memory 管个性化长期事实，Tool 管真实业务动作，三者要分开治理。
- 模型网关负责供应商适配、路由、fallback、限流、熔断、Token 预算、成本归因和观测。
- 评测观测层负责 Trace、日志、指标、Golden Set、LLM-as-Judge、灰度和回放。

## 高频面试题：

- Prompt Demo 到生产系统最大的差距是什么？
- 怎么设计一个生产级 AI 应用的整体架构？
- 一次 AI 请求从入口到模型返回，完整链路应该怎么讲？
- 入口层、编排层、Prompt/Context、RAG/Memory/Tool、模型网关、评测观测分别承担什么职责？
- 同步、流式、异步三种模式怎么选？
- 为什么需要模型网关？
- Prompt 为什么要做版本管理？
- RAG 和 Memory 有什么区别？为什么不能混在一起治理？
- Tool Calling 的安全边界在哪里？

- AI 应用可观测要看哪些指标？
- LLM-as-Judge 能不能替代人工评测？

回答“怎么设计生产级 AI 应用”时，可以用一个通用模板：先说明业务目标和约束，再讲分层架构，然后讲一次请求链路，接着讲稳定性、安全、成本、观测和评测，最后讲灰度和回滚。这样比直接报一堆技术名词更有说服力。

## 稳定性、成本与安全治理

参考文章：《AI 应用系统设计：从 Prompt Demo 到生产级架构》、《大模型 API 调用工程实践：流式输出、重试、限流与结构化返回》

这一组题考的是生产意识。大模型调用慢、贵、不稳定，输出还不可完全控。没有治理能力，AI 应用很容易在上线后变成成本黑洞和事故来源。

建议掌握这些关键点：

- 超时要分层设置：入口超时、模型调用超时、工具调用超时、异步任务超时。
- 重试只适合网络瞬断、部分 5xx、供应商过载等可恢复错误；参数错误、权限错误、安全拒答不能盲目重试。
- 限流要同时看请求数、Token 数、并发数、租户预算和模型供应商配额。
- fallback 要谨慎。模型降级可能影响质量、格式、工具调用能力和安全策略，不是所有任务都能自动降级。
- Token 成本要归因到租户、用户、功能、模型、Prompt 版本和业务场景。
- Tool Calling 安全必须由后端强制执行，不能相信模型自己判断权限。

## 高频面试题：

- AI 应用如何做超时、重试、限流、熔断和降级？
- 为什么大模型调用限流要同时看 RPM、TPM、并发数和租户预算？
- 如何设计模型 fallback 策略？什么时候不能自动降级？
- Token 成本怎么归因到租户、用户、功能和 Prompt 版本？
- 高风险工具调用为什么要做二次确认？
- PII 脱敏、权限过滤、审计日志应该放在哪些环节？
- Prompt 注入攻击在系统设计层面怎么防？
- 出现模型输出事故后，如何通过 Trace 回放定位问题？

回答安全题时，一定要强调：Prompt 只能辅助，不能替代代码层面的权限校验。模型可以建议调用工具，但后端必须校验用户身份、资源归属、参数范围、操作风险和幂等状态。

## 评测与持续迭代

参考文章：《AI 应用评测体系：从 Golden Set 构建到线上灰度闭环》

传统系统上线前可以跑单元测试、集成测试、压测；AI 应用还要评测答案质量、检索质量、工具轨迹和结构化输出稳定性。没有评测闭环，就很难知道一次 Prompt 调整、模型切换、检索参数变化到底是提升还是退步。

建议掌握这些关键点：

- Golden Set 是发布前质量回归的基础，应该覆盖正常路径、边缘场景、对抗样本和高权重失败。
- 离线评测适合发布前阻断明显退步，Trace 回放适合复现真实线上路径，线上灰度适合验证真实用户分布。
- RAG 要分检索和生成评测，Agent 要看任务完成率、工具选择、参数准确率和轨迹质量。
- LLM-as-Judge 可以提高效率，但要用人工抽样、规则校验和参考答案校准。
- 评测结果要和 Prompt 版本、模型版本、检索配置、代码版本绑定，便于回滚和定位。

高频面试题：

- 为什么没有评测集就很难放心上线？
- Golden Set 如何覆盖正常路径、边缘场景、对抗样本和高权重失败？
- 离线评测、Trace 回放、线上灰度分别放在发布流程的哪个阶段？
- RAG、Agent、结构化输出的评测指标为什么不能混用一套？
- LLM-as-Judge 有哪些偏差？生产中怎么校准？
- CI 自动评测怎么控制成本和耗时？
- 线上质量下降时，如何判断是模型、Prompt、检索、工具还是数据分布变化导致？

面试里可以把评测讲成一条流水线：开发阶段跑小规模核心 Golden Set，合并或发布前跑完整评测，灰度阶段做线上抽样，事故后用 Trace 回放复现，失败样本再回流到评测集。

### 实时语音 Agent

参考文章：《AI 语音技术详解：从 ASR、TTS 到实时语音 Agent 的工程化落地》

实时语音 Agent 是很典型的 AI 系统设计题，因为它同时考多模态链路、低延迟、状态机、打断处理和端云选型。

建议掌握这些关键点：

- 语音 Agent 不是 ASR + LLM + TTS 的简单拼接，而是一套实时音频流系统。
- 完整链路包括音频采集、VAD、ASR、LLM、工具调用、TTS、流式播放和打断处理。
- 端到端延迟来自多个环节：音频帧提交、VAD 判断、ASR 转写、LLM 首字、TTS 首包、网络和播放缓冲。
- 打断处理要取消播放、取消生成、处理已播放内容和未播放内容，并更新对话状态。
- 云端 API 上线快，本地模型可控但工程成本高，端云混合更适合兼顾体验和成本。

高频面试题：

- 如何设计一个实时语音 Agent？
- ASR、LLM、TTS、VAD 在语音系统中分别负责什么？
- 实时语音 Agent 的端到端延迟主要来自哪里？
- 用户打断时，系统应该如何取消播放、取消生成和更新上下文？
- listening、thinking、speaking、interrupted 这些状态如何管理？
- 云端 API、本地模型、端云混合怎么选？
- Speech-to-Speech API 适合什么场景？有哪些取舍？
- 语音 Agent 的可观测指标应该包括哪些？

回答实时语音题时，可以先拆链路，再讲低延迟优化，接着讲状态机和打断，最后讲可观测和选型。不要只停留在“调用语音识别和语音合成接口”。

### 系统设计答题模板

遇到开放式 AI 系统设计题，可以按下面顺序回答：

1. **明确场景和约束**：用户规模、响应时延、数据来源、权限要求、成本预算、质量目标。
2. **拆分核心链路**：入口、编排、上下文、RAG、Memory、Tool、模型网关、输出校验、观测评测。
3. **讲关键数据流**：一次请求如何鉴权、检索、组装 Prompt、调用模型、处理流式输出、记录 Trace。
4. **补治理能力**：限流、熔断、重试、幂等、fallback、成本归因、权限控制、审计日志。
5. **讲评测闭环**：Golden Set、离线评测、Trace 回放、线上灰度、失败样本回流。
6. **说明取舍边界**：哪些场景同步，哪些场景流式，哪些场景异步；哪些任务允许降级，哪些必须人工确认。

这套模板能覆盖大多数 AI 应用系统设计题，包括智能客服、企业知识库、代码助手、数据分析 Agent、语音 Agent。

### 常见扣分点

- 上来就讲框架名，不讲业务约束和系统边界。
- 只讲 Prompt 和模型，不讲 RAG、Memory、Tool 的治理差异。
- 没有模型网关意识，业务代码直接调用供应商 API。
- 不记录 Prompt 版本、模型版本、检索结果、工具轨迹，导致事故无法回放。
- 把 LLM-as-Judge 当成万能评测，不做人工校准和规则校验。
- 只靠 Prompt 做安全防护，忽略权限、脱敏、审计和二次确认。
- 没有灰度、回滚和失败样本回流机制。

### 复习建议

AI 系统设计面试要按“系统链路”来回答，不要从某个框架或工具名开始。更稳的表达方式是先讲 Demo 和生产差距，再讲分层架构、核心链路、治理能力和评测闭环。

如果面试官继续追问，再展开模型网关、Prompt 版本、RAG 和 Memory 隔离、Tool Calling 安全、Trace 回放、灰度评测这些关键点。

最后记住一句话：**AI 系统设计不是让模型回答一次，而是让系统长期、稳定、可控地回答。**能把这句话展开成架构、链路、治理和评测，基本就能答到面试官想听的层次。